

Complete Step-by-Step Lab Guide: CrowdStrike Security Scanning + Azure Deployment

What We're Building

We're going to:

1. Create a simple web application
 2. Put it in a container (like a lunch box for apps!)
 3. Scan it for security problems with CrowdStrike
 4. Deploy it to Azure (Microsoft's cloud)
-

Part 1: What You Need Before Starting

Things to Install on Your Computer:

1. **Visual Studio Code** - <https://code.visualstudio.com/>
2. **Docker Desktop** - <https://www.docker.com/products/docker-desktop/>
3. **Azure CLI** - <https://docs.microsoft.com/en-us/cli/azure/install-azure-cli>
4. **Git** - <https://git-scm.com/downloads>

Accounts You Need:

1. **Azure Account** (free trial works!) - <https://azure.microsoft.com/free/>
 2. **Azure DevOps Account** (free!) - <https://dev.azure.com/>
 3. **CrowdStrike Account** (ask your teacher/admin for credentials)
-

Part 2: Create Your Simple Web Application

Step 1: Create a Project Folder

1. Open your computer's file explorer
2. Create a new folder called `my-first-app` on your Desktop
3. Open Visual Studio Code
4. Click `File` → `Open Folder` → Select `my-first-app`

Step 2: Create Your Web Application Files

File 1: Create `app.js` (This is your web application)

```
// This is a simple web server
const express = require('express');
const app = express();
const port = 8080;

// When someone visits our website, show them this message
app.get('/', (req, res) => {
  res.send(`
    <html>
    <head>
      <title>My First App</title>
      <style>
        body {
          font-family: Arial;
          text-align: center;
          padding: 50px;
          background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
          color: white;
        }
        h1 { font-size: 48px; }
        p { font-size: 24px; }
      </style>
    </head>
    <body>
      <h1>🎉 Hello from My First App!</h1>
  `);
});
```

```

    <p>This app was scanned by CrowdStrike and deployed to Azure!</p>
    <p>Build Number: ${process.env.BUILD_ID || 'local'}</p>
  </body>
</html>
  `);
});

// Health check endpoint (so Azure knows our app is working)
app.get('/health', (req, res) => {
  res.status(200).send('OK');
});

// Start the server
app.listen(port, () => {
  console.log(`🚀 App is running on port ${port}`);
});

```

File 2: Create `package.json` (This tells Node.js what our app needs)

```

{
  "name": "my-first-app",
  "version": "1.0.0",
  "description": "My first containerized app with CrowdStrike scanning",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}

```

File 3: Create `Dockerfile` (This is the recipe to package our app)

```

# Start with a base image that has Node.js
FROM node:18-alpine

```

```
# Create a folder for our app
WORKDIR /app

# Copy our package.json file
COPY package.json .

# Install the things our app needs
RUN npm install

# Copy all our app files
COPY . .

# Tell Docker our app uses port 8080
EXPOSE 8080

# Command to start our app
CMD ["npm", "start"]
```

File 4: Create `.dockerignore` (Things Docker should ignore)

```
node_modules
npm-debug.log
.git
.gitignore
README.md
.env
```

Step 3: Test Your App Locally (Optional but Fun!)

1. Open Terminal in VS Code (View → Terminal)
2. Type these commands one by one:

```
# Install Node.js packages
npm install
```

```
# Run your app  
npm start
```

1. Open your web browser and go to: `http://localhost:8080`
2. You should see your colorful webpage! 🎉
3. Press `Ctrl+C` in the terminal to stop the app

☁️ Part 3: Create Azure Resources

Step 1: Login to Azure

1. Open Terminal/Command Prompt
2. Type:

```
az login
```

1. A web browser will open - login with your Azure account
2. Close the browser when it says "You can close this window"

Step 2: Create a Resource Group (Like a folder for all your Azure stuff)

```
az group create \\  
  --name my-first-app-rg \\  
  --location eastus
```

What this does: Creates a container to hold all your Azure resources

Step 3: Create Azure Container Registry (ACR) - Where we store our app images

```
az acr create \\  
  --resource-group my-first-app-rg \\  
  --name my-first-app-registry
```

```
--name myfirstappacr2024 \  
--sku Basic \  
--location eastus
```

Important: Write down the name `myfirstappacr2024` - you'll need it later!

Note: If you get an error saying the name is taken, try: `myfirstappacr2024yourname`

Step 4: Create Azure Kubernetes Service (AKS) - Where our app will run

```
az aks create \  
--resource-group my-first-app-rg \  
--name my-first-app-aks \  
--node-count 2 \  
--node-vm-size Standard_B2s \  
--enable-managed-identity \  
--generate-ssh-keys \  
--location eastus
```

What this does: Creates a cluster of computers to run your app

Time: This takes about 5-10 minutes 🕒 (Go get a snack!)

Step 5: Connect ACR to AKS (So AKS can get images from ACR)

```
az aks update \  
--resource-group my-first-app-rg \  
--name my-first-app-aks \  
--attach-acr myfirstappacr2024
```

Replace `myfirstappacr2024` with your ACR name if different

Step 6: Get Your ACR Login Server Address

```
az acr show \  
--name myfirstappacr2024 \  
--resource-group my-first-app-rg
```

```
--query loginServer \\  
--output tsv
```

Write this down! It will look like: `myfirstappacr2024.azurecr.io`

Part 4: Setup Azure DevOps

Step 1: Create Azure DevOps Organization

1. Go to <https://dev.azure.com/>
2. Click "Start free"
3. Sign in with your Microsoft account
4. Create a new organization (name it anything, like "MyFirstOrg")

Step 2: Create a New Project

1. Click "New Project"
2. Name: `my-first-app`
3. Visibility: Private
4. Click "Create"

Step 3: Create a Git Repository

1. In your project, click "Repos" on the left
2. You'll see instructions to push code
3. Keep this page open - we'll use it soon!

Step 4: Push Your Code to Azure DevOps

1. Open Terminal in VS Code (in your `my-first-app` folder)
2. Type these commands one by one:

```
# Initialize git  
git init
```

```
# Add all your files
```

```
git add .
```

```
# Save your files with a message
```

```
git commit -m "My first app ready for deployment"
```

```
# Connect to Azure DevOps (replace with YOUR URL from Step 3)
```

```
git remote add origin <https://dev.azure.com/YourOrg/my-first-app/_git/my-first-app>
```

```
# Push your code
```

```
git push -u origin --all
```

Replace `YourOrg` with your organization name!

Step 5: Create Service Connections

A. Create Azure Service Connection

1. In Azure DevOps, click "Project Settings" (bottom left)
2. Click "Service connections"
3. Click "New service connection"
4. Select "Azure Resource Manager" → Click "Next"
5. Select "Service principal (automatic)" → Click "Next"
6. Fill in:
 - Subscription: Select your Azure subscription
 - Resource group: `my-first-app-rg`
 - Service connection name: `azure-connection`
 - Check "Grant access permission to all pipelines"
7. Click "Save"

B. Create ACR Service Connection

1. Click "New service connection" again
2. Select "Docker Registry" → Click "Next"
3. Select "Azure Container Registry"
4. Fill in:
 - Subscription: Select your Azure subscription
 - Azure container registry: Select `myfirstappacr2024`
 - Service connection name: `acr-connection`
 - Check "Grant access permission to all pipelines"
5. Click "Save"

Step 6: Setup Self-Hosted Agent (This runs your pipeline)

Install Agent on Your Computer:

For Windows:

1. In Azure DevOps, click "Project Settings" → "Agent pools"
2. Click "Default" pool
3. Click "New agent"
4. Click "Download" for Windows
5. Extract the zip file to `C:\agent`
6. Open PowerShell as Administrator
7. Run:

```
cd C:\agent
.\config.cmd
```

1. When asked:
 - Server URL: `https://dev.azure.com/YourOrg`
 - PAT token: (We'll create this next)

- Agent pool: `Default`
 - Agent name: `my-agent`
 - Work folder: Press Enter (default)
2. Run as service? Type `Y` and press Enter
 3. Run:

```
.\run.cmd
```

For Mac/Linux:

1. In Azure DevOps, click "Project Settings" → "Agent pools"
2. Click "Default" pool
3. Click "New agent"
4. Download for your OS
5. Extract to `~/agent`
6. Open Terminal and run:

```
cd ~/agent  
./config.sh
```

1. Follow same prompts as Windows
2. Run:

```
./run.sh
```

Create Personal Access Token (PAT):

1. Click your profile picture (top right)
2. Click "Personal access tokens"
3. Click "New Token"
4. Name: `agent-token`

5. Expiration: 90 days
 6. Scopes: Select "Full access"
 7. Click "Create"
 8. **COPY THE TOKEN** - you can't see it again!
 9. Use this token when configuring the agent
-

Part 5: Setup CrowdStrike Variables

Step 1: Get CrowdStrike Credentials

Ask your CrowdStrike administrator for:

- Client ID
- Client Secret

Step 2: Add Variables to Azure DevOps

1. In Azure DevOps, go to "Pipelines" → "Library"
 2. Click "+" to create a new Variable Group
 3. Name it: `crowdstrike-credentials`
 4. Add these variables:
 - Name: `FALCON_CLIENT_ID` , Value: (your client ID)
 - Name: `FALCON_CLIENT_SECRET` , Value: (your client secret), **Check the lock icon**

 5. Click "Save"
-

Part 6: Create the Pipeline

Step 1: Create Pipeline File

1. In VS Code, create a new file called `azure-pipelines.yml`
2. Copy this complete pipeline:

```

trigger:
  - main

pool:
  name: 'Default' # Your self-hosted agent pool

variables:
  imageName: 'my-first-app'
  imageTag: '$(Build.BuildId)'
  CS_REGISTRY: "registry.crowdstrike.com"
  CS_IMAGE_NAME: "registry.crowdstrike.com/fcs/eu-1/release/cs-fcs"
  CS_USERNAME: "fh-39AECBE16A5D4284B1E412F3AC267D18"
  CS_IMAGE_TAG: 2.1.0
  FALCON_API_URL: "<https://api.eu-1.crowdstrike.com>"
  FALCON_REGION: "eu-1"
  # Update these with YOUR values
  aksResourceGroup: 'my-first-app-rg'
  aksClusterName: 'my-first-app-aks'
  acrName: 'myfirstappacr2024' # CHANGE THIS to your ACR name
  acrLoginServer: 'myfirstappacr2024.azurecr.io' # CHANGE THIS to your AC
R login server
  k8sNamespace: 'default'
  deploymentName: 'my-first-app'

stages:
  - stage: Build
    displayName: '🔨 Build and Scan'
    jobs:
      - job: BuildAndScan
        displayName: 'Build Docker Image and Security Scan'
        steps:
          - script: |
              echo "🎨 Building Docker image..."
              docker build . --tag $(imageName):$(imageTag) --tag $(imageName):
latest

```

```

    echo "✅ Image built successfully: ${imageName}:${imageTag}"
    displayName: "Build container image"

- script: |
    set -e
    echo "🔑 Getting Falcon API Access Token..."
    export FALCON_ACCESS_TOKEN=$(curl --silent --request POST \\\
    --header "Content-Type: application/x-www-form-urlencoded" \\\
    --data-urlencode "client_id=${FALCON_CLIENT_ID}" \\\
    --data-urlencode "client_secret=${FALCON_CLIENT_SECRET}" \\\
    --url "${FALCON_API_URL}/oauth2/token" | jq -r '.access_token')

    if [ -z "$FALCON_ACCESS_TOKEN" ] || [ "$FALCON_ACCESS_TOKEN"
= "null" ]; then
        echo "❌ Failed to retrieve Falcon API access token"
        exit 1
    fi

    echo "✅ Falcon API token retrieved successfully"

    echo "🔑 Getting CrowdStrike registry password..."
    export CS_PASSWORD=$(curl --silent --request GET \\\
    --header "Authorization: Bearer ${FALCON_ACCESS_TOKEN}" \\\
    --url "${FALCON_API_URL}/iac/entities/image-registry-credentials/v
1" | jq -r '.resources.resources.token')

    if [ -z "$CS_PASSWORD" ] || [ "$CS_PASSWORD" = "null" ]; then
        echo "❌ Failed to retrieve CrowdStrike registry password"
        exit 1
    fi

    echo "✅ Registry password retrieved successfully"
    echo "##vso[task.setvariable variable=CS_PASSWORD]$CS_PASSW
ORD"

    displayName: "🔑 Get Falcon Credentials"
    env:

```

```
FALCON_CLIENT_ID: $(FALCON_CLIENT_ID)
FALCON_CLIENT_SECRET: $(FALCON_CLIENT_SECRET)
```

```
- script: |
  echo "🔍 Verifying built image exists..."
  docker images | grep $(imageName)
  if [ $? -eq 0 ]; then
    echo "✅ Image $(imageName):$(imageTag) found successfully"
  else
    echo "❌ Error: Image not found"
    exit 1
  fi
  displayName: "Verify image exists"
```

```
- script: |
  set +x
  scan_status=0
```

```
echo "🛡️ Starting CrowdStrike Security Scan..."
```

```
if [[ -z "$CS_USERNAME" || -z "$CS_PASSWORD" || -z "$CS_REGISTRY" ]]; then
  echo "❌ Error: Missing required credentials"
  exit 1
fi
```

```
echo "🔑 Logging in to CrowdStrike registry..."
echo "$CS_PASSWORD" | docker login "$CS_REGISTRY" --username
"$CS_USERNAME" --password-stdin
```

```
if [ $? -eq 0 ]; then
  echo "✅ Docker login successful"
  echo "📦 Pulling FCS scanner image..."
```

```
docker pull "$CS_IMAGE_NAME":"$CS_IMAGE_TAG"
```

```

if [ $? -eq 0 ]; then
    echo "✅ FCS scanner image pulled successfully"
    echo "🔍 Scanning image: $(imageName):$(imageTag)"

    echo "===== 🛡️ FCS Image Scan Starts =====
=====
    docker run --rm \\\
    -v /var/run/docker.sock:/var/run/docker.sock \\\
    $CS_IMAGE_NAME:$CS_IMAGE_TAG scan image $(imageName)
e):$(imageTag) \\\
    --client-id $FALCON_CLIENT_ID \\\
    --client-secret $FALCON_CLIENT_SECRET \\\
    --falcon-region $FALCON_REGION \\\
    --upload \\\
    --log-level debug \\\
    --fail-on medium

    scan_status=$?
    echo "===== ✅ FCS Image Scan Ends =====
=====
    echo "📊 Scan status code: ${scan_status}"
    echo $scan_status > fcs_scan_status.txt
else
    echo "❌ Error: Failed to pull FCS scanner image"
    exit 2
fi
else
    echo "❌ Error: Docker login failed"
    exit 3
fi
displayName: "🛡️ Run CrowdStrike Security Scan"
env:
    FALCON_CLIENT_ID: $(FALCON_CLIENT_ID)
    FALCON_CLIENT_SECRET: $(FALCON_CLIENT_SECRET)
    CS_PASSWORD: $(CS_PASSWORD)

```

```

- script: |
  scan_status=$(cat fcs_scan_status.txt)
  echo "📊 Scan result: $scan_status"

  if [ "$scan_status" -eq 40 ]; then
    echo "⚠️ Vulnerabilities found above threshold"
    echo "###vso[task.complete result=SucceededWithIssues;]Vulnerabi
ities detected"
  elif [ "$scan_status" -eq 0 ]; then
    echo "✅ No critical vulnerabilities found!"
  else
    echo "❌ Unexpected scan error: $scan_status"
    exit 1
  fi
  displayName: "📊 Evaluate Scan Results"

- script: |
  echo "✅ Security scan complete!"
  echo "📦 Image: $(imageName):$(imageTag)"
  echo "🏗️ Build ID: $(Build.BuildId)"
  displayName: "✅ Scan Complete"
  condition: always()

- stage: Deploy
  displayName: '🚀 Deploy to Azure'
  dependsOn: Build
  condition: succeeded()
  jobs:
    - job: DeployToAKS
      displayName: 'Deploy to Kubernetes'
      steps:
        - task: Docker@2
          displayName: '🏷️ Tag image for ACR'
          inputs:
            command: 'tag'
            arguments: '$(imageName):$(imageTag) $(acrLoginServer)/$(imageName)'

```



```
ame):$(imageTag)'
```

- task: Docker@2
 displayName: '🔑 Login to ACR'
 inputs:
 command: 'login'
 containerRegistry: 'acr-connection'

- task: Docker@2
 displayName: '📦 Push image to ACR'
 inputs:
 command: 'push'
 repository: '\$(imageName)'
 containerRegistry: 'acr-connection'
 tags: |
 \$(imageTag)
 latest

- task: AzureCLI@2
 displayName: '🔑 Get AKS credentials'
 inputs:
 azureSubscription: 'azure-connection'
 scriptType: 'bash'
 scriptLocation: 'inlineScript'
 inlineScript: |
 echo "🔑 Getting AKS credentials..."
 az aks get-credentials \\
 --resource-group \$(aksResourceGroup) \\
 --name \$(aksClusterName) \\
 --overwrite-existing
 echo "✅ Credentials retrieved"

- script: |
 echo "📝 Creating Kubernetes deployment manifest..."
 cat <<EOF > deployment.yaml
 apiVersion: apps/v1

```
kind: Deployment
metadata:
  name: $(deploymentName)
  namespace: $(k8sNamespace)
spec:
  replicas: 2
  selector:
    matchLabels:
      app: $(deploymentName)
  template:
    metadata:
      labels:
        app: $(deploymentName)
    spec:
      containers:
        - name: $(deploymentName)
          image: $(acrLoginServer)/$(imageName):$(imageTag)
          ports:
            - containerPort: 8080
          env:
            - name: BUILD_ID
              value: "$(Build.BuildId)"
      resources:
        requests:
          memory: "128Mi"
          cpu: "100m"
        limits:
          memory: "256Mi"
          cpu: "200m"
      livenessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
```

```

    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 5
  ---
  apiVersion: v1
  kind: Service
  metadata:
    name: $(deploymentName)-service
    namespace: $(k8sNamespace)
  spec:
    type: LoadBalancer
    ports:
      - port: 80
        targetPort: 8080
    selector:
      app: $(deploymentName)
EOF

echo "✅ Manifest created"
cat deployment.yaml
displayName: "📝 Create Kubernetes Manifest"

- script: |
  echo "🚀 Deploying to Kubernetes..."
  kubectl apply -f deployment.yaml
  echo "✅ Deployment command executed"
  displayName: "🚀 Deploy to AKS"

- script: |
  echo "⌚ Waiting for deployment to complete..."
  kubectl rollout status deployment/$(deploymentName) -n $(k8sName
space) --timeout=5m

echo ""

```

```

echo "📊 Deployment Status:"
kubectl get deployment $(deploymentName) -n $(k8sNamespace)

echo ""
echo "🎯 Pods:"
kubectl get pods -n $(k8sNamespace) -l app=$(deploymentName)

echo ""
echo "🌐 Service:"
kubectl get service $(deploymentName)-service -n $(k8sNamespac
e)

echo ""
echo "🎉 Getting your app's public IP address..."
echo "⌚ This might take a few minutes..."

for i in {1..30}; do
    EXTERNAL_IP=$(kubectl get service $(deploymentName)-service -n
$(k8sNamespace) -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
    if [ ! -z "$EXTERNAL_IP" ]; then
        echo ""
        echo "🎉🎉🎉 SUCCESS! 🎉🎉🎉"
        echo "Your app is live at: http://$EXTERNAL_IP"
        echo "Open this URL in your browser!"
        break
    fi
    echo "Still waiting for IP address... ($i/30)"
    sleep 10
done

if [ -z "$EXTERNAL_IP" ]; then
    echo "⚠️ IP address not ready yet. Run this command later:"
    echo "kubectl get service $(deploymentName)-service -n $(k8sNa
mespace)"

```

```
fi
displayName: "✅ Verify Deployment & Get URL"
```

Step 2: Update the Pipeline with YOUR Values

In the `azure-pipelines.yml` file, find these lines and update them:

```
acrName: 'myfirstappacr2024' # Change to YOUR ACR name
acrLoginServer: 'myfirstappacr2024.azurecr.io' # Change to YOUR ACR login
server
```

Step 3: Save and Push to Azure DevOps

```
git add azure-pipelines.yml
git commit -m "Add pipeline configuration"
git push
```

Part 7: Run Your Pipeline!

Step 1: Create the Pipeline in Azure DevOps

1. Go to Azure DevOps → Your project
2. Click "Pipelines" on the left
3. Click "New Pipeline"
4. Select "Azure Repos Git"
5. Select your repository `my-first-app`
6. Select "Existing Azure Pipelines YAML file"
7. Path: `/azure-pipelines.yml`
8. Click "Continue"

Step 2: Link Variable Group

1. Before running, click "Variables" (top right)

2. Click "Variable groups"
3. Click "Link variable group"
4. Select `crowdstrike-credentials`
5. Click "Link"
6. Click "Save"

Step 3: Run the Pipeline!

1. Click "Run" (big blue button)
2. Click "Run" again to confirm
3. Watch the magic happen! 🎉

What You'll See:

- 🏗️ **Build Stage:** Building your app and scanning for security issues
- 🛡️ **CrowdStrike Scan:** Checking for vulnerabilities
- 🚀 **Deploy Stage:** Pushing to Azure and deploying to Kubernetes

🎉 Part 8: See Your App Running!

Step 1: Get Your App's URL

After the pipeline finishes (about 5-10 minutes), look at the last step "Verify Deployment & Get URL"

You'll see something like:

🎉🎉🎉 SUCCESS! 🎉🎉🎉

Your app is live at: <http://20.123.45.67>

Step 2: Open Your App

1. Copy that IP address
2. Open your web browser

3. Paste the IP address
4. Press Enter
5. **YOU DID IT!** 🎉 You should see your colorful webpage!

Step 3: Check CrowdStrike Results

1. Go to your CrowdStrike console
 2. Navigate to: **Cloud Security** → **Image Assessment**
 3. Look for your image: `my-first-app`
 4. Click on it to see all the security scan results!
-

Part 9: Troubleshooting (If Something Goes Wrong)

Problem: Pipeline fails at "Build container image"

Solution:

- Make sure Docker Desktop is running on your computer
- Check that your self-hosted agent is running

Problem: Pipeline fails at "Get Falcon Credentials"

Solution:

- Check your CrowdStrike credentials in the Variable Group
- Make sure they're not expired
- Verify the Client ID and Secret are correct

Problem: Pipeline fails at "Deploy to AKS"

Solution:

- Check your service connections are set up correctly
- Verify your ACR name and login server are correct in the YAML file
- Make sure AKS cluster is running:

```
az aks show --resource-group my-first-app-rg --name my-first-app-aks
```

Problem: Can't access the app URL

Solution:

- Wait a few more minutes (Azure needs time to assign the IP)
- Check if the service is running:

```
kubectl get service my-first-app-service -n default
```

- Look for `EXTERNAL-IP` - if it says `<pending>`, wait a bit longer

Problem: No vulnerabilities showing in CrowdStrike

Solution:

- Check the pipeline logs for "Upload successful"
- Verify you're looking at the correct CrowdStrike region (EU-1)
- Wait a few minutes - results can take time to appear
- Check that `-upload` flag is in the scan command

Part 10: Understanding What You Built

What is Docker?

Think of Docker like a lunch box. You put your app and everything it needs (like ingredients) into the lunch box. Then you can take that lunch box anywhere and it will work the same way!

What is Kubernetes (AKS)?

Kubernetes is like a smart robot that manages your lunch boxes (containers). If one breaks, it makes a new one. If you need more, it creates more. It's very smart!

What is CrowdStrike?

CrowdStrike is like a security guard that checks your lunch box to make sure there's nothing bad inside before you use it.

What is Azure DevOps?

Azure DevOps is like an automatic factory. You give it instructions (the pipeline), and it automatically builds, tests, and deploys your app!

Part 11: What to Do Next

Make Changes to Your App

1. Edit `app.js` - change the message or colors
2. Save the file
3. Push to Azure DevOps:

```
git add .  
git commit -m "Updated my app"  
git push
```

1. Watch the pipeline run automatically!
2. Refresh your browser to see the changes!

Clean Up (When You're Done)

To avoid Azure charges, delete everything:

```
az group delete --name my-first-app-rg --yes --no-wait
```

Cheat Sheet - Quick Commands

Check if your app is running:

```
kubectl get pods -n default
```

See your app's URL:

```
kubectl get service my-first-app-service -n default
```

See app logs:

```
kubectl logs -l app=my-first-app -n default
```

Restart your app:

```
kubectl rollout restart deployment/my-first-app -n default
```

Congratulations!

You just:

-  Created a web application
-  Containerized it with Docker
-  Scanned it for security with CrowdStrike
-  Deployed it to Azure Kubernetes
-  Set up automated CI/CD with Azure DevOps

You're now a DevOps engineer!   

Need Help?

If you get stuck:

1. Read the error message carefully
2. Check the troubleshooting section
3. Look at the pipeline logs in Azure DevOps
4. Ask your teacher/mentor
5. Google the error message

Remember: Every expert was once a beginner. You're doing great! ✨